

## Taller #3

**Desarrollado por: Lenin Barrios, Andrés García y Alberto Márquez**

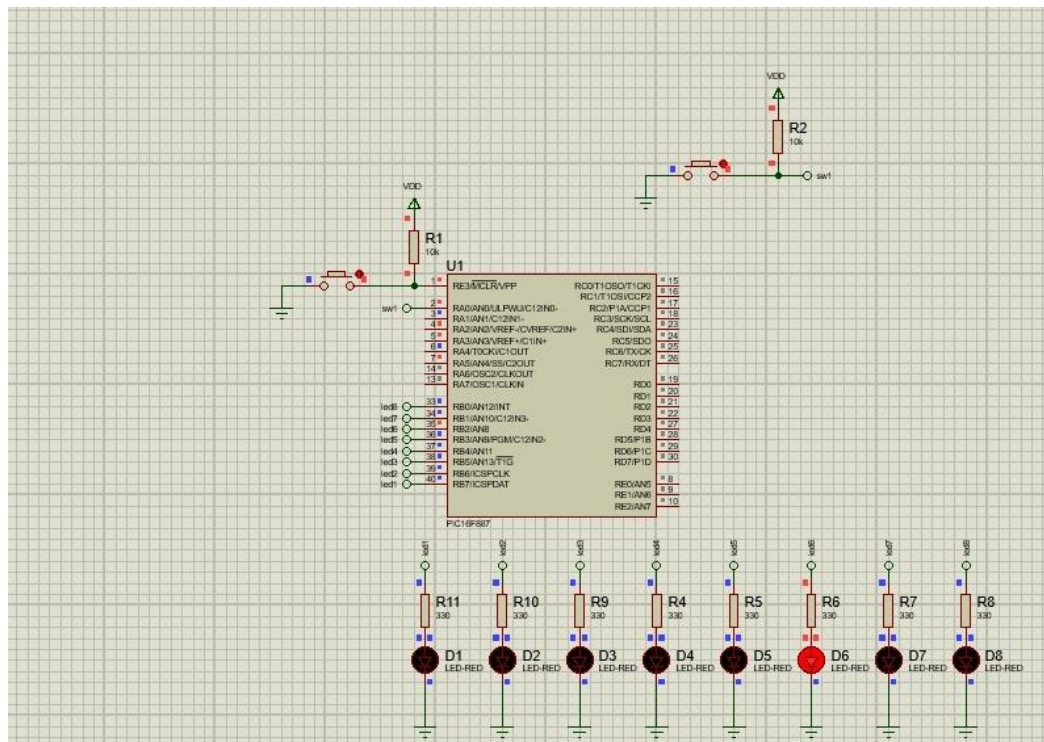
## Ejercicio 1

Para el desarrollo de este ejercicio nos pedían diseñar un sistema con el PIC16F887 que controlara 8 leds mediante un pulsador, permitiendo alternar entre cuatro secuencias diferentes de desplazamiento. La primera secuencia desplaza un led encendido de derecha a izquierda, la segunda de izquierda a derecha, la tercera desplaza dos leds desde el centro hacia los extremos, y la cuarta desplaza dos leds desde los extremos hacia el centro. Cada led debe permanecer encendido por medio segundo antes de pasar a la siguiente posición.

- **Enlace del video**

<https://www.youtube.com/shorts/6OUuJ> FC-eM

- **Desarrollo de la simulación del ejercicio en Proteus**



- **Código fuente:**

```
#include <xc.h>
#include <stdint.h>
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = OFF
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF
#define _XTAL_FREQ 4000000
void main(void) {
    TRISB = 0x00;
    TRISA = 0x01;
    PORTB = 0x00;
    uint8_t modo = 0;
    uint8_t paso = 0;
    uint8_t btn_ant = 0;
    while (1) {
        uint8_t btn = RA0;

        if (btn == 1 && btn_ant == 0) {
            modo++;
            if (modo > 2) modo = 0;
            paso = 0;
        }
        btn_ant = btn;

        switch (modo) {
            case 0:
                PORTB = 1 << paso;
                paso++;
                if (paso >= 8) paso = 0;
                break;
            case 1:
                PORTB = 1 << (7 - paso);
                paso++;
                if (paso >= 8) paso = 0;
                break;
            case 2:
                switch (paso) {
                    case 0: PORTB = 0x18;
                    break;
                    case 1: PORTB = 0x24;
                    break;
                    case 2: PORTB = 0x42;
                    break;
                    case 3: PORTB = 0x81;
                    break;
                }
                paso++;
                if (paso >= 4) paso = 0;
            }
    }
}
```

```

break;
}
__delay_ms(3000);
}
}

```

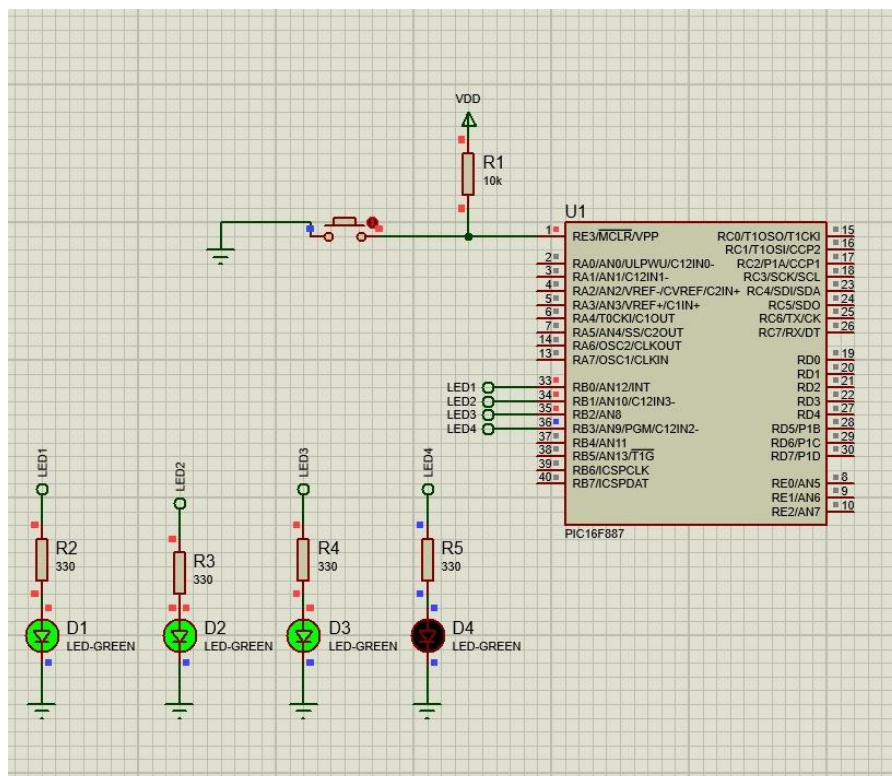
## Ejercicio 2

Para el desarrollo de este ejercicio nos pedían diseñar un sistema con el PIC16F887, el cual mostraría una secuencia de código Gray de 4 bits usando una serie de 4 leds.. Lo que teníamos que hacer era programar el microcontrolador para que recorriera los 16 estados del código Gray (de 0000 a 1111), mostrando cada patrón en los leds durante 1 segundo antes de cambiar al siguiente, y repetir el ciclo indefinidamente.

- **Enlace del video:**

<https://www.youtube.com/shorts/OP-P4Wo9rhk>

- **Desarrollo de la simulación del ejercicio en Proteus**



- **Código fuente:**

```
punto 2
#include <xc.h>

#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = OFF
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF

#define _XTAL_FREQ 2000000UL

int main(void)
{
    TRISB  = 0xF0;
    PORTB  = 0x00;

    unsigned char n = 0;

    while (1)
    {
        unsigned char gray = n ^ (n >> 1);
        PORTB = (PORTB & 0xF0) | (gray & 0x0F);
        __delay_ms(1000);
        n = (n + 1) & 0x0F;
    }
}
```

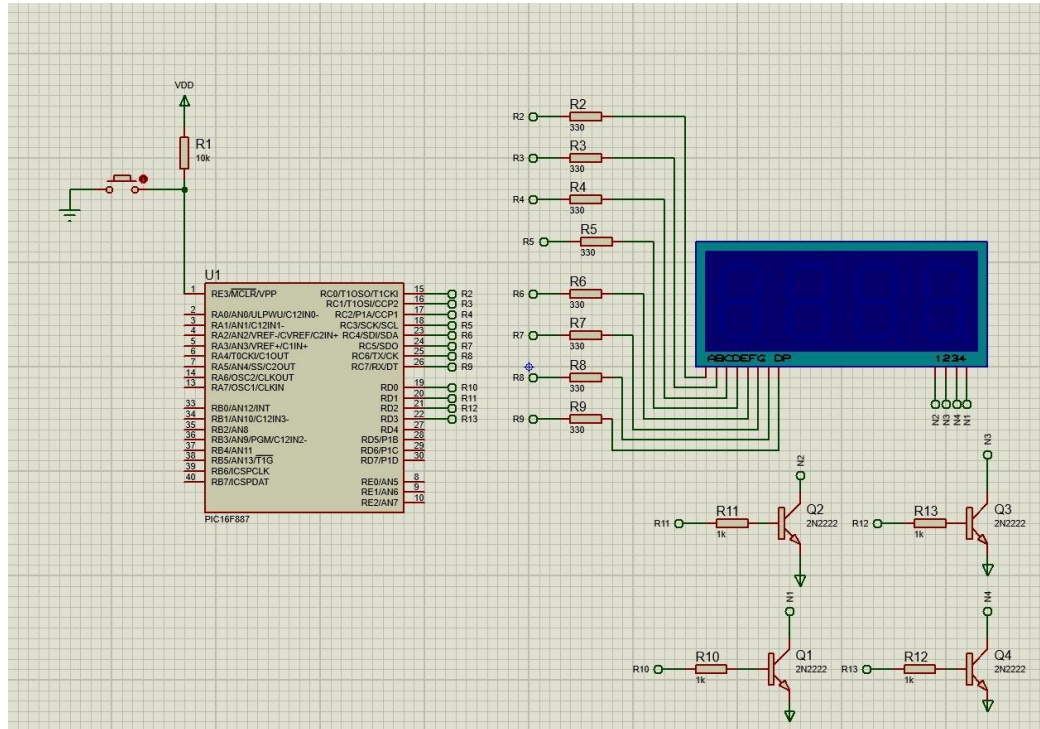
### **Ejercicio 3**

Para el desarrollo de este ejercicio nos pidieron crear un sistema el cual nos permita realizar un conteo desde el 0000 hasta el 9999, para que sea visible este conteo utilizamos un display 7 segmentos de cuatro dígitos.

- **Enlace del video:**

<https://www.youtube.com/watch?v=D6CCkPFnZTg>

- **Desarrollo de la simulación del ejercicio en Proteus**



- **Código fuente:**

```
punto 3
#include <xc.h>
#define _XTAL_FREQ 4000000

#pragma config FOSC = INTRC_NOCLKOUT
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = OFF
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF

const unsigned char digitos[10] = {
    0x3F,
    0x06,
    0x5B,
    0x4F,
    0x66,
    0x6D,
    0x7D,
    0x07,
    0x7F,
```

```

        0x6F
    };

static inline void mostrarNumero(unsigned int n) {
    unsigned char u = n % 10;
    unsigned char d = (n / 10) % 10;
    unsigned char c = (n / 100) % 10;
    unsigned char m = (n / 1000) % 10;

    const unsigned char val[4] = {m, c, d, u};
    const unsigned char sel[4] = {0x01, 0x02, 0x04, 0x08};

    for (unsigned char k = 0; k < 4; k++) {
        PORTD = 0x00;
        PORTC = digitos[val[k]];
        PORTD = sel[k];
        __delay_ms(2);
    }
}

void main(void) {

    OSCCON = 0b01100001;

    ANSEL = 0x00;
    ANSELH = 0x00;

    TRISC = 0x00;
    TRISD = 0x00;
    PORTC = 0x00;
    PORTD = 0x00;

    unsigned int contador = 0;

    while (1) {

        for (unsigned char i = 0; i < 50; i++) {
            mostrarNumero(contador);
        }

        contador++;
        if (contador > 9999) contador = 0;
    }
}

```

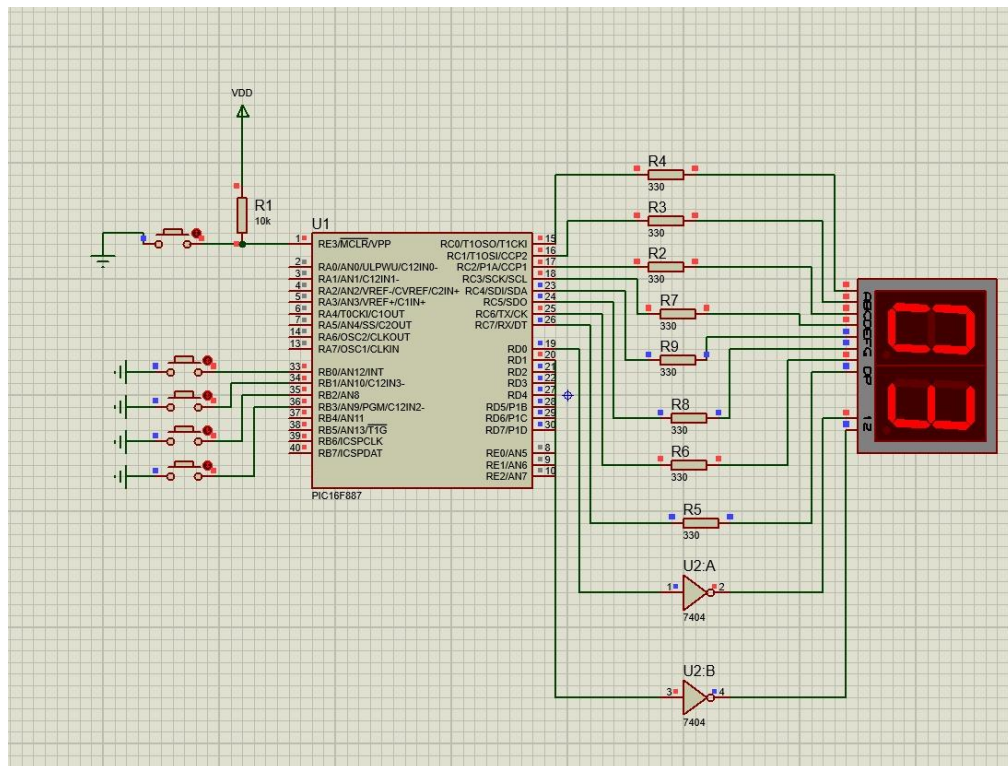
## Ejercicio 4

Para este ejercicio nos pedían desarrollar un cronómetro digital controlado por pulsadores y mostrado en displays de 7 segmentos. El sistema debía contar desde 00 hasta 99 segundos y tener cuatro funciones principales mediante botones: iniciar la cuenta, pausarla, reanudarla y reiniciar a cero. Para el desarrollo del proyecto nos pidieron que montáramos todo el circuito dentro de una baquela de cobre, lo que implicaba diseñar las pistas de conexión, soldar los componentes (PIC16F887, displays, resistencias limitadoras, pulsadores), y asegurar un montaje funcional y ordenado.

- **Enlace del video:**

<https://www.youtube.com/shorts/tYHeTGbgKg0>

- **Desarrollo de la simulación del ejercicio en Proteus**



- **Código fuente**

```
punto4
#include <xc.h>

#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = OFF
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF

#define _XTAL_FREQ 20000000
#define TIEMPO 40

#define BTN_START    RB0
#define BTN_PAUSE    RB1
#define BTN_RESUME    RB2
#define BTN_RESET    RB3

const unsigned char DIGITOS[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D,
0x7D, 0x07, 0x7F, 0x6F};

void mostrar_numeros(int valor)
{
    unsigned char aux = 0;
    unsigned char dec = 0;
    unsigned char uni = 0;

    aux = valor / 100;
    dec = (valor - aux * 100) / 10;
    uni = valor - aux * 100 - dec * 10;

    PORTC = DIGITOS[uni];
    PORTD = 0x02;
    __delay_ms(TIEMPO);
    PORTD = 0x00;

    PORTC = DIGITOS[dec];
    PORTD = 0x01;
    __delay_ms(TIEMPO);
    PORTD = 0x00;
}

int main(void)
{
    unsigned char cont = 0;
    int valor = 0;

    ANSEL = 0x00;
    ANSELH = 0x00;
```

```

PORTB = 0xFF;
TRISB = 0x0F;
OPTION_REGbits.nRBPU = 0;
WPUB = 0x0F;

unsigned char prev_start = 1, prev_pause = 1, prev_resume = 1,
prev_reset = 1;
unsigned char running = 0;

PORTC = 0x00;
TRISC = 0x00;

PORTD = 0x00;
TRISD = 0x00;

while(1)
{
    mostrar_numeros(valor);

    if (BTN_START == 0 && prev_start == 1) {
        running = 1;
    }
    if (BTN_PAUSE == 0 && prev_pause == 1) {
        running = 0;
    }
    if (BTN_RESUME == 0 && prev_resume == 1) {
        running = 1;
    }
    if (BTN_RESET == 0 && prev_reset == 1) {
        valor = 0;
        cont = 0;
    }

    prev_start = BTN_START;
    prev_pause = BTN_PAUSE;
    prev_resume = BTN_RESUME;
    prev_reset = BTN_RESET;

    if (running) {
        cont++;
        if(cont == 12)
        {
            cont = 0;
            valor++;
            if(valor == 100)
            {
                valor = 0;
            }
        }
    }
}

return 0;
}

```

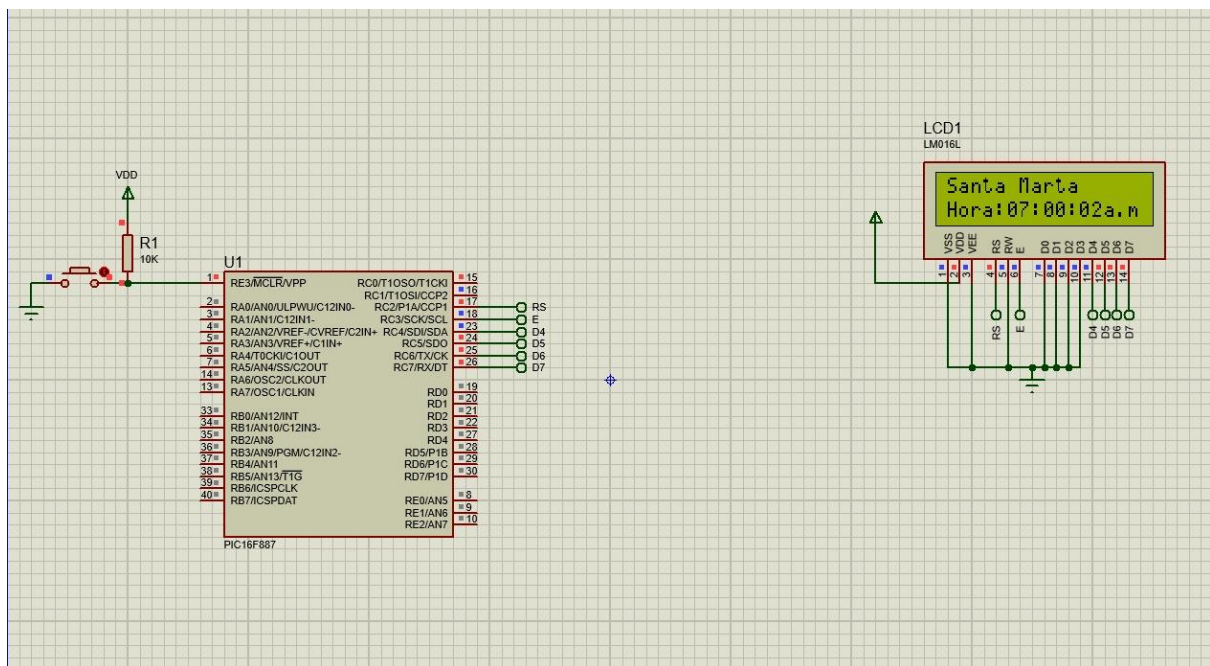
## Ejercicio 5

Para este ejercicio nos pidieron desarrollar un sistema que nos permita visualizar en una LCD 16x2 la ciudad en donde nos encontramos y una hora de manera completamente aleatoria la cual debe ir incrementando su valor durante su simulación. Por ejemplo: Santa Marta, hora: 07:00 a.m. -> Santa Marta, hora: 07:02 a.m.

- **Enlace del video:**

<https://www.youtube.com/watch?v=JGMcIua5R1A>

- **Desarrollo de la simulación del ejercicio en Proteus**



- **Código fuente**

#1

```
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include "lcd_i2cd.h"

#define _XTAL_FREQ 2000000UL

#pragma config FOSC = HS
#pragma config WDTE = OFF
```

```

#pragma config PWRTE = OFF
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = OFF
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF

#define SIM_MS_PER_SEC 200

typedef struct {
    uint8_t hour;
    uint8_t min;
    uint8_t sec;
    bool    is_am;
} Clock12;

static void clock_init(Clock12* c, uint8_t h, uint8_t m, uint8_t s, bool
am) {
    c->hour   = (h >= 1 && h <= 12) ? h : 12;
    c->min    = (m <= 59) ? m : 0;
    c->sec    = (s <= 59) ? s : 0;
    c->is_am = am;
}

static void clock_tick_sec(Clock12* c) {
    if (++c->sec >= 60) {
        c->sec = 0;
        if (++c->min >= 60) {
            c->min = 0;
            if (++c->hour > 12) c->hour = 1;
            if (c->hour == 12) c->is_am = !c->is_am;
        }
    }
}

static void clock_format16(const Clock12* c, char out16[17]) {
    sprintf(out16, "Hora:%02u:%02u:%02u%s",
        c->hour, c->min, c->sec, c->is_am ? "am" : "pm");

    uint8_t i = 0;
    while (out16[i] != '\0' && i < 16) i++;
    for (; i < 16; i++) out16[i] = ' ';
    out16[16] = '\0';
}

void main(void) {

    ANSEL  = 0x00;
    ANSELH = 0x00;

    I2C_Init();
    LCD_Init();

```

```

const char line1[] = "Santa Marta";

Clock12 clk;
clock_init(&clk, 7, 1, 0, true);

char line2[17];

while (1) {

    LCD_Set_Cursor(0, 0);
    LCD_putrs("                ");
    LCD_Set_Cursor(0, 0);
    LCD_putrs((char*)line1);

    clock_format16(&clk, line2);
    LCD_Set_Cursor(1, 0);
    LCD_putrs(line2);

    __delay_ms(SIM_MS_PER_SEC);
    clock_tick_sec(&clk);
}
}

```

## #2

```

lcd_i2cd.h

#ifndef LCD_I2C_H
#define LCD_I2C_H

#include <stdbool.h>
#include <stdint.h>

/* ==== API I2C ==== */
void I2C_Init(void);

/* ==== API LCD ==== */
bool LCD_Init(void);
void LCD_putc(char a);
void LCD_puts(char *a);
void LCD_putrs(const char *a);
void LCD_Cmd(uint8_t c);

#define LCD_Clear() LCD_Cmd(0x01)
#define LCD_Home() LCD_Cmd(0x02)
#define LCD_DisplayOn() LCD_Cmd(0x0C)
#define LCD_DisplayOff() LCD_Cmd(0x08)
#define LCD_CursorOn() LCD_Cmd(0x0E)
#define LCD_BlinkOn() LCD_Cmd(0x0D)
#define LCD_Shift_Right() LCD_Cmd(0x1C)

```

```

#define LCD_Shift_Left()          LCD_Cmd(0x18)

#define LCD_Set_Cursor(x,y)      \
do {                             \
    uint8_t pos = (x)==0 ? (0x80+(y)) : (0xC0+(y)); \
    LCD_Cmd(pos);                \
} while(0)

#endif

```

### #3

```

#include <xc.h>
#include "lcd_i2cd.h"

#define _XTAL_FREQ 2000000UL

#define LCD_ADDR 0x7E
#define I2C_BAUD 100000UL

#define RS 0x01
#define RW 0x02
#define EN 0x04
#define BL 0x08

/* ===== I2C PRIMITIVAS ===== */
static void I2C_Start(void) { SEN = 1; while(SEN); }
static void I2C_Stop(void) { PEN = 1; while(PEN); }
static void I2C_Write(uint8_t d) {
    SSPBUF = d;
    while(BF);
    while(SSPSTATbits.R_nW);
}

void I2C_Init(void) {

    TRISCbits.TRISC3 = 1;
    TRISCbits.TRISC4 = 1;

    SSPCON = 0x28;
    SSPCON2 = 0x00;
    SSPADD = (uint8_t)(((_XTAL_FREQ/4)/I2C_BAUD) - 1);
    SSPSTAT = 0x00;
}

static void lcd_write_byte(uint8_t data) {
    I2C_Start();
    I2C_Write(LCD_ADDR);
    I2C_Write(data | BL);
    I2C_Stop();
}

```

```

static void lcd_pulse_enable(uint8_t data) {
    // EN=1
    lcd_write_byte(data | EN);
    __delay_us(1);
    // EN=0
    lcd_write_byte(data & ~EN);
    __delay_us(40);
}

static void LCD_Send(uint8_t val, uint8_t rs) {
    uint8_t high = (val & 0xF0);
    uint8_t low  = ((val << 4) & 0xF0);
    if (rs) { high |= RS; low |= RS; }

    lcd_pulse_enable(high);
    lcd_pulse_enable(low);
}

void LCD_Cmd(uint8_t c) {
    LCD_Send(c, 0);
    if (c == 0x01 || c == 0x02) __delay_ms(2);
}

static void LCD_Char(uint8_t c) {
    LCD_Send(c, 1);
}

bool LCD_Init(void) {
    __delay_ms(20);

    LCD_Cmd(0x33);
    LCD_Cmd(0x32);

    LCD_Cmd(0x28);

    LCD_Cmd(0x0C);

    LCD_Cmd(0x06);
    // Clear
    LCD_Clear();

    return true;
}

void LCD_putc(char a) { LCD_Char((uint8_t)a); }
void LCD_puts(char *s) { while (*s) LCD_putc(*s++); }
void LCD_putstr(const char *s) { while (*s) LCD_putc(*s++); }

```